

JxAgent: Regression-Aware Post-Training for Reliable Long-Horizon Computer Use Agents

Technical White Paper

Julius Pfaff

Independent Researcher, Germany

jxagent18@gmail.com

<https://github.com/JxAgentReal/JxAgent>

August 2026

Version 0.1. This manuscript documents the current research design and implementation. No final model-quality result is claimed.

Abstract

Computer-use agents have improved quickly, but long-horizon desktop reliability remains fragile. A model can perform well on isolated actions and still fail after an ineffective click, repeat the same operation, mishandle a modal dialog, save to the wrong location, or stop without checking whether the requested state was actually reached. Improving these behaviors through post-training creates a second problem: a narrow specialization run can damage capabilities that were already strong in the base model.

JXAGENT is an experimental framework for studying that trade-off. The current Run 1 design targets Qwen/Qwen3.8-27B and combines a failure-focused multimodal training mixture with an explicit regression guard. The central hypothesis is that a relatively small, carefully selected specialization dataset, paired with checkpoint-by-checkpoint preservation evaluation, can improve difficult computer-use behavior more safely than broad task-domain fine-tuning. The planned Run 1 mixture contains 95,503 samples drawn from public computer-use trajectory and grounding resources plus a bounded preservation replay cohort. Training uses LoRA with a frozen vision stack and a model-native interaction contract that must be derived from the exact pinned model revision before any reportable run.

The evaluation protocol compares the base model and every candidate checkpoint on identical task IDs, preprocessing, tool schemas, generation settings, and frozen preservation samples. Checkpoint selection is treated as a constrained optimization problem over computer-use gain, capability preservation, and serving cost rather than as a default preference for the final training step. This paper describes the motivation, data pipeline, native-interface contract, training protocol, evaluation methodology, reproducibility requirements, and planned ablations. It intentionally reports no benchmark improvement before controlled GPU experiments are complete.

Keywords: computer-use agents; multimodal agents; post-training; LoRA; regression evaluation; long-horizon reliability; OSWorld

1 Introduction

A useful computer-use agent must do more than choose a plausible next click. It has to preserve task state over many steps, recover when an action has no effect, notice when an application

opens a dialog, verify that a save or export actually occurred, avoid loops, and know when the task is finished. These failures are especially important in desktop environments because a single mistaken action can alter the state seen by every later step.

OSWorld made this problem concrete by introducing a real-computer benchmark with 369 tasks spanning web and desktop applications, file operations, and cross-application workflows [2]. More recent datasets have expanded the available supervision. GUI-360 provides large-scale desktop trajectories and grounding data [5]; CUA-Suite contributes continuous human demonstration video and dense grounding annotations [6]; PC Agent-E studies how a relatively small amount of high-quality trajectory data can support strong computer-use training [4]; and ProCUA-SFT scales synthetic desktop trajectories substantially further [7].

The availability of more data does not remove the adaptation problem. In fact, recent computer-use work provides a direct warning: ProCUA-SFT reports that continuing supervised training on a different large trajectory corpus produced severe negative transfer on OSWorld for the model studied there [7]. That observation motivates the central premise of JXAGENT: for a strong base model, specialization should be measured as a controlled behavioral change, not merely as additional training.

The project therefore asks a narrower question than “can more desktop data improve a model?” The question is whether difficult computer-use failure modes can be improved while keeping the rest of the model measurably intact. This changes the engineering priorities. Data quality is necessary but not sufficient. Training and evaluation must share the same native interaction format, intermediate checkpoints must be retained, preservation tasks must be frozen before the run, and the final model should be selected from a gain-versus-regression frontier rather than by training progress alone.

2 Research Problem and Design Principles

Let M_0 denote the pinned base model and M_θ an adapted checkpoint produced by parameter-efficient post-training. Let $G(\theta)$ represent improvement on the target computer-use suite, $P_k(\theta)$ the change on preservation capability k , and $C(\theta)$ the serving or compute cost associated with the checkpoint. The model-selection objective can be written as

$$\theta^* \in \arg \max_{\theta} G(\theta) \quad \text{subject to} \quad P_k(\theta) \geq -\epsilon_k \quad \forall k, \quad C(\theta) \leq C_{\max}. \quad (1)$$

The tolerances ϵ_k are not assumed to be identical across capabilities. Safety behavior, native tool formatting, and instruction hierarchy may require stricter bounds than a noisy secondary metric. The point of the formulation is not the exact optimization algorithm; it is the decision rule. A checkpoint with a larger target-domain gain is not automatically preferable if it causes systematic regressions elsewhere.

Three principles follow from this objective. First, the base model is treated as a valuable prior rather than a blank starting point. Second, training supervision should be concentrated on failure modes that are plausibly underrepresented in generic instruction tuning. Third, every reportable improvement must be paired with preservation evidence collected under the same experimental contract.

3 System Overview

The JXAGENT pipeline is organized around a sequence of explicit gates. Upstream computer-use datasets are read through source-specific adapters, normalized into a common trajectory representation, filtered for structural and semantic quality, decontaminated against held-out evaluation instructions, and selected into a bounded training mixture. The exact model interface is then frozen from the pinned model and processor revisions. LoRA training is allowed to begin only after encoding, label masking, action parsing, and generation-prefix tests pass. Intermediate checkpoints are evaluated against both target tasks and preservation canaries before any production-scale run is accepted.

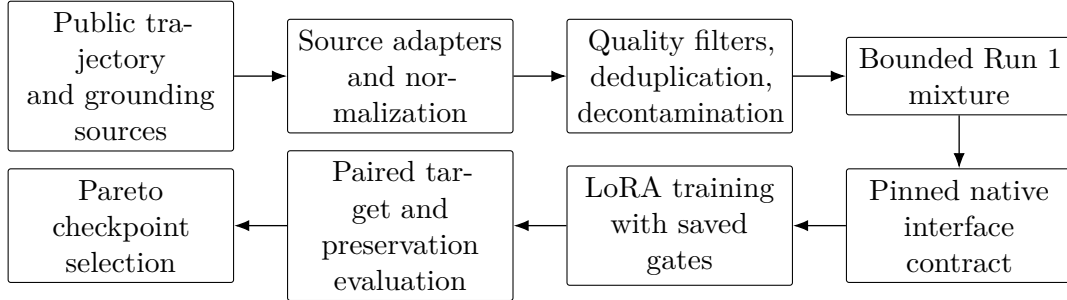


Figure 1: High-level JXAGENT Run 1 workflow. No model-quality claim is produced until the full path from data construction to paired evaluation has passed its gates.

This separation is deliberate. Infrastructure checks such as valid action syntax, correct coordinate conversion, or successful checkpoint reload are prerequisites for trustworthy evaluation, but they are not themselves evidence that the model improved.

4 Data Construction

4.1 Run 1 mixture

The planned Run 1 dataset contains 95,503 multimodal samples. It is intentionally smaller than several recent computer-use corpora because the experiment focuses on controlled specialization and regression mapping rather than maximum-scale continued training. The current target mixture is shown in Table 1.

Table 1: Planned JxAgent Run 1 mixture. Counts are selection targets, not claims about the full size of the upstream datasets.

Source	Target samples	Primary role
NVIDIA ProCUA-SFT [7]	46,000	Desktop action trajectories
GUI-360 [5]	16,000	Grounding and screen understanding
VideoCUA / CUA-Suite [6]	17,500	Temporal desktop behavior
GroundCUA / CUA-Suite [6]	4,000	UI grounding
PC Agent-E [4]	4,503	Desktop trajectories
Preservation replay mixture	7,500	General capability preservation
Total	95,503	

The replay component is bounded by design. The current target allocation is 1,600 coding samples, 1,500 mathematics samples, 1,700 general instruction samples, 1,400 visual-question-answering

samples, and 1,300 native tool-style samples. These counts are only a first-order control. Before production training, the project requires exact supervised-token accounting because sample counts do not directly measure gradient contribution.

4.2 Failure-focused supervision

The training target is not generic desktop competence. Run 1 prioritizes cases where a capable model may still be unreliable: recovery after ineffective actions, evidence-gated finishing, loop avoidance, dialog and modal handling, save and export workflows, exact-quantity tasks, sorting and ranking, multi-target tasks, multi-application workflows, and precise GUI grounding.

Recovery supervision is constructed only when the observed state supports a real transition. A repeated screenshot followed by repeated actions should not be labeled as successful recovery merely because the source trajectory continued. Temporal windows are therefore checked for state changes, action diversity, and excessive waiting. Grounding samples are validated after final-image coordinate normalization, with out-of-bounds targets rejected and trusted bounding boxes used when available.

Run 1 sets the synthetic reasoning rate to zero. This is a conservative choice: the objective is to supervise observable action policy, state-sensitive correction, and verifiable completion behavior without inventing hidden chains of reasoning that cannot be independently checked.

4.3 Deduplication, split integrity, and contamination control

Computer-use datasets are vulnerable to near-duplicate screenshots, templated tasks, and semantically repeated trajectories. JXAGENT applies context-aware perceptual deduplication and group-aware splitting so that related examples do not leak across train and validation partitions. Split auditing compares source distribution, task motif, action type, trajectory length, image count, and supervised-token distribution rather than relying only on sample totals.

OSWorld and OSWorld Verified are kept out of the Run 1 training mixture. Their task instructions may be used for contamination detection, but evaluation trajectories are not copied into training. External benchmark evaluation is scheduled only after the training manifest and checkpoint-selection rules are frozen.

5 The Native Interaction Contract

A computer-use model is unusually sensitive to interface mismatches. The same underlying action can be represented as text, a structured tool call, normalized coordinates, pixel coordinates, or a model-specific action grammar. A training pipeline can therefore produce apparently valid examples that are still wrong for the target model.

JXAGENT addresses this by treating the interaction interface as an executable experiment artifact. The current public configuration names `Qwen/Qwen3.8-27B` as the target, but the model identifier alone is not considered sufficient provenance. Before any reportable run, the exact model revision, processor revision, chat template, tool schema, image placement behavior, action serialization, coordinate convention, history format, finish semantics, thinking policy, and generation prefix must be derived from the pinned local artifacts and frozen into a hashed contract.

This requirement is intentionally stricter than copying conventions from an earlier model in the same family. Qwen3 provides useful architectural and training context for the broader family [3], but JXAGENT does not infer the behavior of the configured target from earlier Qwen releases. If

the model or processor revision changes, the previous contract is invalidated and the interface tests must be rerun.

The contract is verified through golden examples that exercise encoding, label masks, parser round trips, coordinate conversion, tool syntax, finish behavior, and the assistant generation prefix. Training and evaluation use the same frozen contract so that a measured difference is attributable to the adapter rather than to a hidden change in prompting or serialization.

6 Training Protocol

6.1 Parameter-efficient adaptation

The initial training plan uses Low-Rank Adaptation (LoRA) [1]. LoRA is appropriate for the first experiment because it limits the number of trainable parameters and makes checkpoint-level ablation practical. The vision encoder, vision-language aligner, embeddings, and language-model head are intended to remain frozen. Candidate trainable modules in the text decoder are selected only after inspecting the exact pinned architecture.

Table 2 summarizes the current preproduction configuration. These values are an experimental starting point rather than a claim of optimal hyperparameters.

Table 2: Initial Run 1 training configuration. Architecture-dependent target modules are resolved only after model inspection.

Setting	Current plan
Method	LoRA
LoRA rank / alpha	32 / 64
LoRA dropout	0.0
Learning rate	1×10^{-5}
Scheduler	Cosine with 0.03 warmup ratio
Precision	bfloat16
Maximum sequence length	8,192 tokens
Packing	Disabled
Gradient checkpointing	Enabled
Per-device batch size	1
Global effective batch	Approximately 32 via dynamic gradient accumulation
Epochs	1 for the initial controlled run
Checkpoint policy	Multiple retained gates; no destructive rotation during training

The project also audits trainable parameter mass and update norms by module family. This is important if the pinned architecture contains recurrent-state or memory-gating projections: such modules may have a disproportionate effect on long-horizon behavior and are therefore treated as an explicit ablation target rather than silently included in a broad “all linear layers” rule.

6.2 Staged compute use

The compute plan is intentionally staged. The first phase verifies the accelerator and software environment, performs a three-step training smoke test, checks checkpoint reload, confirms that only intended LoRA parameters update, and measures bounded throughput. The second phase

trains on 5,000 selected preproduction samples while retaining multiple checkpoints. Only if this run shows useful movement on the target behaviors without unacceptable preservation loss does the project proceed to larger ablations and the production mixture.

The repository includes an AMD MI300X and ROCm workflow, while the same experimental contract is designed to be portable to NVIDIA cloud GPUs. Cross-accelerator runs are useful only if the dataset manifest, native contract, precision, training semantics, and evaluation protocol remain fixed enough to make the comparison meaningful.

7 Regression-Aware Evaluation

7.1 Paired evaluation

Every base-versus-adapter comparison uses the same task IDs, preprocessing, native interface, generation settings, and environment configuration. This pairing matters because aggregate scores can hide systematic regressions if the two models are evaluated on different subsets or under slightly different scaffolds.

For binary task outcomes, the planned report includes the paired difference in success rate, bootstrap confidence intervals over task IDs, and a paired significance check such as McNemar’s test when sample size is appropriate. For continuous metrics, paired bootstrap intervals are preferred over independent-sample intervals. These statistical procedures are reporting tools; they do not replace practical regression thresholds defined before training.

7.2 Target metrics

Computer-use evaluation covers end-to-end task success together with failure-specific diagnostics. These include action validity, coordinate validity, recovery success after ineffective actions, repeated-action loop rate, premature finishing, evidence before finish, and cross-application reliability. A single aggregate benchmark score is useful but insufficient for identifying why an adapter changed.

OSWorld and OSWorld Verified are reserved for held-out external evaluation after the data and checkpoint-selection policy are frozen. The project also maintains smaller frozen diagnostic subsets for fast checkpoint sweeps. Results from infrastructure tests or development subsets are never labeled as full benchmark results.

7.3 Preservation suite

The preservation side of the experiment includes general instruction following, coding, mathematics and scientific reasoning, multilingual behavior, native tool use, document and OCR understanding, charts, visual mathematics, natural-image reasoning, safety and refusal calibration, ordinary conversational style, and long-context retrieval and instruction retention.

These canaries are frozen before the run and evaluated sample by sample. If a regression appears, the preferred response is diagnosis: identify the responsible training cohort, module family, or training stage. The protocol explicitly avoids reflexively adding large quantities of replay data, because a larger replay mixture can obscure the cause of the regression and reduce the target signal.

7.4 Checkpoint selection

Early, middle, and final checkpoints are preserved, and additional checkpoints may be retained when the compute budget allows. The best model is selected from the empirical frontier between target improvement, preservation delta, and serving cost. This is one of the central departures from ordinary fine-tuning practice: the end of the epoch is a candidate checkpoint, not a privileged checkpoint.

8 Planned Ablations

The first ablations are chosen to answer concrete causal questions rather than to search a large hyperparameter grid. The recovery-cohort ablation asks whether explicit supervision on real ineffective-action transitions reduces loops and increases successful recovery. The verification ablation tests whether evidence-gated finish examples improve completion calibration without producing excessive reluctance to finish. The replay ablation measures how much preservation data is actually needed once contribution is measured by supervised loss tokens rather than by raw sample count. The LoRA target-family ablation tests whether restricting or expanding particular decoder module families changes the gain-versus-regression frontier.

Only after those experiments would alternate ranks or learning rates be justified. This ordering is deliberate. When compute is limited, an ablation should distinguish between competing explanations, not merely produce another checkpoint.

9 Reproducibility and Engineering Safeguards

A reportable run records the exact base-model revision, processor revision, source-code commit, interface-contract hash, dataset manifest, split identifiers, random seed, trainer command, software versions, accelerator type, precision, and checkpoint identity. The repository contains deterministic manifest generation, dataset schema tests, source-specific parsing tests, coordinate and image tests, contamination checks, checkpoint loading tests, evaluation dry runs, and public-repository verification tooling.

The training pipeline is designed for faithful resume. Optimizer, scheduler, and random-number-generator state are retained rather than saving model weights alone. Checkpoint rotation is disabled during the critical run so that an automatic cleanup step cannot delete the checkpoint that later turns out to lie on the best preservation frontier.

The public repository is available at <https://github.com/JxAgentReal/JxAgent>. The paper corresponds to the initial public Run 1 design and should be interpreted together with the exact commit and manifests cited in any later results release.

10 Relation to Prior Work

The project sits at the intersection of three lines of work. The first is realistic computer-use evaluation, represented by OSWorld [2]. The second is large-scale or high-quality computer-use supervision, including GUI-360 [5], CUA-Suite [6], PC Agent-E [4], and ProCUA-SFT [7]. The third is parameter-efficient model adaptation, for which LoRA provides the baseline mechanism used in Run 1 [1].

JXAGENT does not propose a new foundation model architecture or a new benchmark. Its contribution is methodological: a regression-aware post-training protocol for adapting an already

capable multimodal model to difficult desktop behavior. The project emphasizes exact interface parity, measured supervised-token contribution, frozen paired preservation evaluation, checkpoint-level regression mapping, and decision rules that allow an earlier checkpoint to beat a later one when the later model has drifted too far.

11 Limitations

The most important limitation is that this paper precedes the main GPU experiments. The 95,503-sample mixture, training configuration, and ablation priorities are hypotheses encoded in a reproducible pipeline, not evidence that the adapted model will improve. Any future result should therefore distinguish clearly between implemented infrastructure, preproduction diagnostics, and held-out benchmark performance.

The source mixture may still contain biases or blind spots that survive automated filtering. Professional applications with rare stateful workflows are difficult to cover. Coordinate correctness does not guarantee semantic correctness. A model can also learn to imitate the surface pattern of verification without actually improving state estimation. These risks require trajectory-level evaluation rather than token-level loss alone.

A second limitation is that capability preservation cannot be proven by a finite canary suite. The regression panel can detect known classes of drift but cannot establish that every general capability remains unchanged. The project therefore treats preservation as empirical evidence with explicit scope, not as a universal guarantee.

Finally, reproducibility across accelerator stacks may be imperfect even with fixed seeds and data. Kernel implementations, attention backends, distributed reduction order, and library versions can introduce small numerical differences. Cross-accelerator replication should focus on whether the scientific conclusion is stable rather than whether every floating-point value is identical.

12 Responsible Use and Data Governance

Computer-use models can automate actions that affect files, accounts, and external services. Research evaluation should therefore use controlled environments, scoped credentials, and reversible tasks whenever possible. The project is intended for reliability research, not for bypassing access controls or automating unauthorized activity.

JXAGENT does not redistribute the upstream datasets or base-model weights. The builder constructs a local derived training set from upstream sources, and users remain responsible for reviewing current licenses and terms before use or redistribution. Benchmark evaluation data is kept separate from the training mixture. Any release of adapted weights should re-check the exact base-model license and all relevant upstream obligations at the revision actually used for training.

13 Current Status and Reporting Policy

At the time of this white paper, the public repository contains the data builder, quality and deduplication pipeline, decontamination logic, native-interface tooling, training configuration, MI300X workflow, evaluation scaffold, regression guard, and automated tests. The main controlled training run has not yet produced a final reportable model result.

This status is intentional. A future result table should not be backfilled from development

observations or infrastructure smoke tests. Positive and negative runs should be published with the same level of configuration detail, including cases where a specialization checkpoint improves computer use but fails the preservation gate. Such negative results are directly relevant to the research question.

14 Conclusion

JXAGENT is built around a simple constraint: improving computer use is not enough if the adaptation quietly damages the model that made the agent useful in the first place. Run 1 therefore combines failure-focused multimodal supervision with a frozen native interface, exact label accounting, staged LoRA training, paired preservation canaries, and checkpoint selection on a gain-versus-regression frontier.

The value of the project will ultimately be determined by controlled experiments rather than by the complexity of the pipeline. The immediate next step is the 5,000-sample preproduction run, followed by checkpoint-level regression mapping. If that stage demonstrates measurable computer-use improvement without unacceptable capability loss, the same protocol can support targeted ablations and a full Run 1 evaluation. If it does not, the failure itself provides a useful measurement of how quickly narrow specialization begins to trade off against a strong base model.

References

- [1] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*, 2021.
- [2] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. *arXiv preprint arXiv:2404.07972*, 2024.
- [3] Qwen Team. Qwen3 Technical Report. *arXiv preprint arXiv:2505.09388*, 2025.
- [4] Yanheng He, Jiahe Jin, and Pengfei Liu. Efficient Agent Training for Computer Use. *arXiv preprint arXiv:2505.13909*, 2025.
- [5] Jian Mu, Chaoyun Zhang, Chiming Ni, Lu Wang, Bo Qiao, Kartik Mathur, Qianhui Wu, Yuhang Xie, Xiaojun Ma, Mengyu Zhou, Si Qin, Liqun Li, Yu Kang, Minghua Ma, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. GUI-360: A Comprehensive Dataset and Benchmark for Computer-Using Agents. *arXiv preprint arXiv:2511.04307*, 2025.
- [6] Xiangru Jian, Shravan Nayak, Kevin Qinghong Lin, Aarash Feizi, Kaixin Li, Patrice Bechard, Spandana Gella, and Sai Rajeswar. CUA-Suite: Massive Human-annotated Video Demonstrations for Computer-Use Agents. *arXiv preprint arXiv:2603.24440*, 2026.
- [7] Jaehun Jung, Ximing Lu, Brandon Cui, Muhammad Khalifa, Shaokun Zhang, Hao Zhang, Jin Xu, Amala Sanjay Deshmukh, Karan Sapra, Andrew Tao, Yejin Choi, Jan Kautz, Mingjie Liu, and Yi Dong. ProCUA-SFT Technical Report. *arXiv preprint arXiv:2606.17321*, 2026.